

Directory Brute-forcing

Cybersecurity Internship – Phase 1, Task 5

Name: Parmar Rakesh

Date: 18, AUG, 2025

Internship Program: Cybersecurity Internship – UJAR TECH

What is a web directory?

Web directories are hierarchical structures that organise web content into categories and subcategories, making it easier for users to navigate and locate the information they need. Directories are distinct from search engines[\[20\]](#), which rely on algorithms to index web content and rank it based on relevance to search queries. In a web directory, each category and subcategory has a unique URL or web address. For example, the URL for the “Sports” category on a hypothetical web directory might be . Within the Sports category, there may be subcategories such as “Basketball”, “Football”, and “Tennis”, each with its own unique URL.

What is Directory Brute-forcing?

Directory brute-forcing involves systematically checking a **list of potential directories** and file names against a web server to discover hidden or non-publicly linked content. The goal is to uncover resources that may contain sensitive information or vulnerabilities, which are not meant to be **publicly accessible** but are inadequately protected.

How does the code work?

Tools like Gobuster, Dirb, and FFuF all work on the same basic mechanism: they take a wordlist of common directory or file names, attach each word to the target URL, and send requests to the server. By analyzing the server’s responses such as status codes (**200 for found, 403 for forbidden, 404 for not found**[\[21\]](#)) or differences in response size the tool identifies which paths actually exist. To make the process efficient, these tools use multi-threading to send many requests in parallel and apply filters to reduce false positives. In short, they automate the process of guessing and checking hidden files or folders by rapidly testing large numbers of possibilities and highlighting the interesting results.

What is GoBuster?

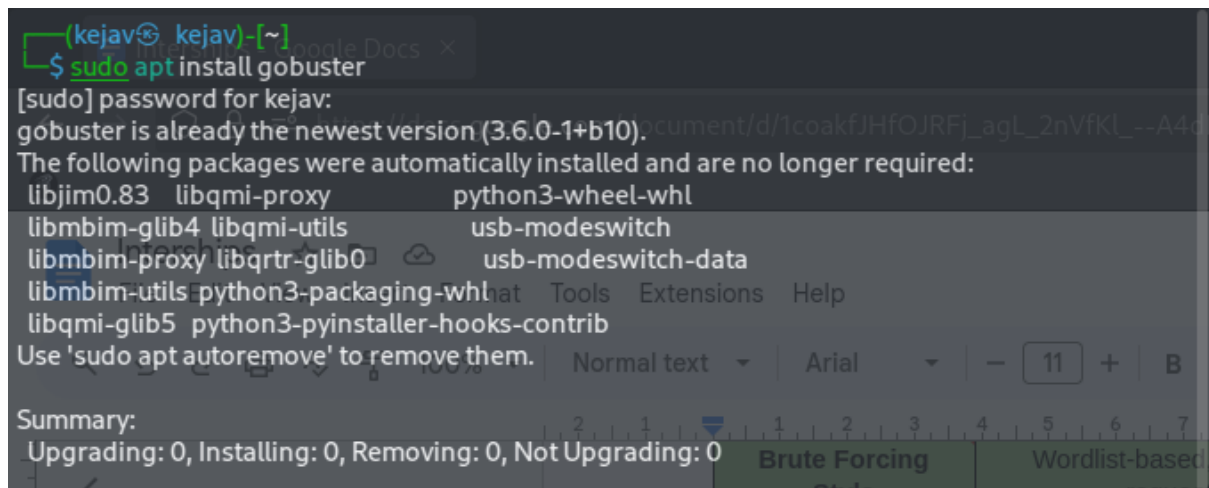
Gobuster is a tool used to find hidden directories, files, DNS subdomains, and virtual hosts on a website. It works using a brute-force approach, meaning it tries many possibilities from a wordlist in a trial-and-error way, which makes it very fast but also “loud” and easier to detect by security systems like IDS. Since it does not automatically search inside discovered directories, you need to run it again on those paths if you want deeper results. Overall, Gobuster is a useful tool for the initial stages of penetration testing to quickly map out a website’s structure and potential hidden areas.

Gobuster vs DIRB

Feature	Gobuster	DIRB
Language	Written in Go (very fast)	Written in C (slower compared to Go)
Brute Forcing Style	Wordlist-based, very fast requests	Wordlist-based, moderate speed
Recursive Scan	Not recursive (needs manual re-run)	Supports recursive scanning with -r
Detection	Loud and easily detected by IDS	Also loud, but slightly slower
Extra Features	Can also brute-force DNS & VHosts	Focused mainly on directories/files
Best Use Case	Quick, large-scale directory scans	Deeper recursive directory exploration

How to install GoBuster?

First, we need to install the tool itself. Just open your terminal and run **sudo apt install gobuster**.



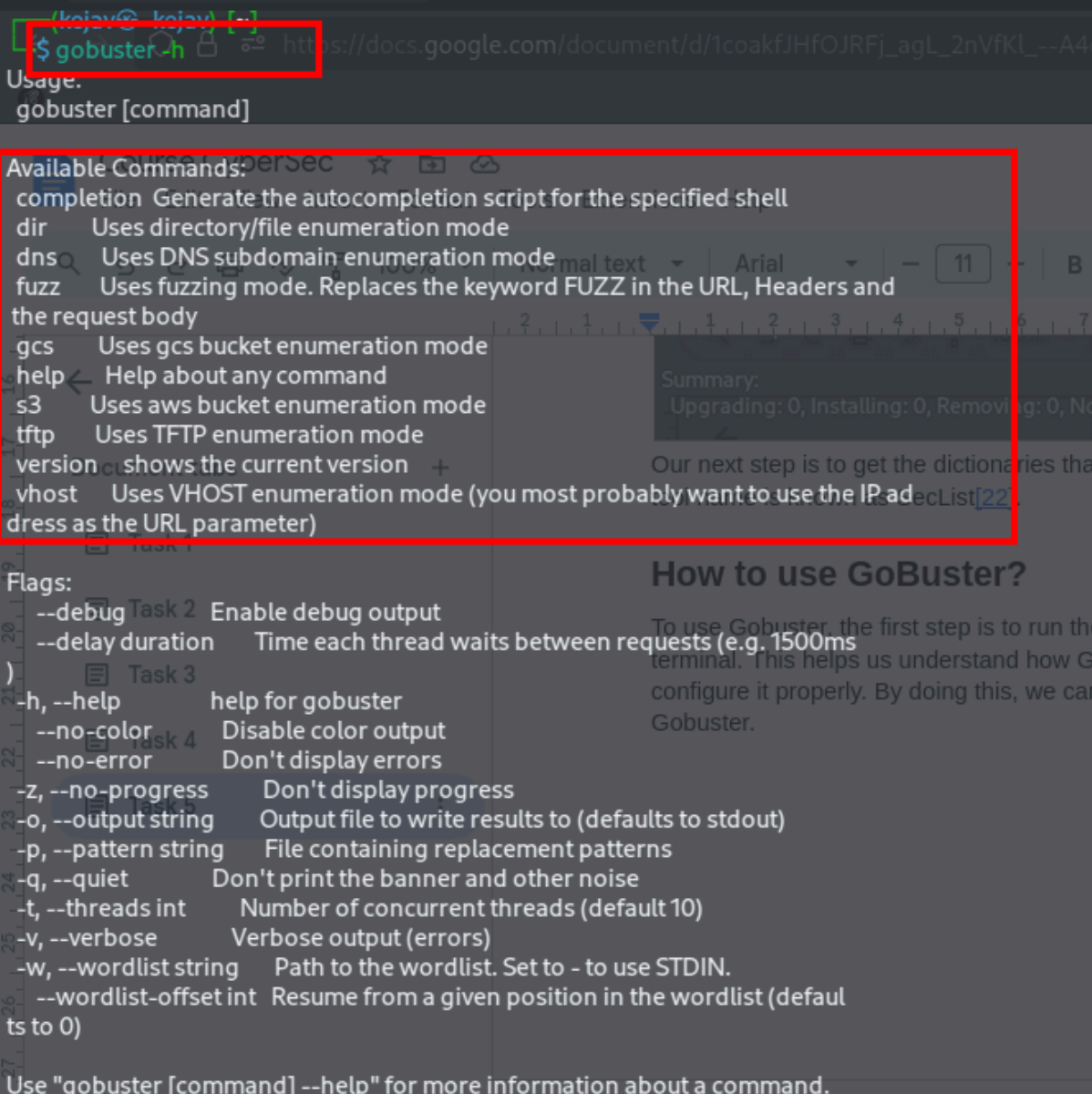
```
(kejav@kejav)~$ sudo apt install gobuster
[sudo] password for kejav:
gobuster is already the newest version (3.6.0-1+b10).
The following packages were automatically installed and are no longer required:
libjim0.83 libqmi-proxy python3-wheel-whl
libmbim-glib4 libqmi-utils usb-modeswitch
libmbim-proxy libqtr-glib0 usb-modeswitch-data
libmbim-utils python3-packaging-whl
libqmi-glib5 python3-pyinstaller-hooks-contrib
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

Our next step is to get the dictionaries that we will use to populate our enumeration which tool name is known as SecList[\[22\]](#).

How to use GoBuster?

To use Gobuster, the first step is to run the **gobuster -h** or **man gobuster** command in the terminal. This helps us understand how Gobuster works, which flags to use, and how to configure it properly. By doing this, we can easily learn the usage and options available in Gobuster.



```
(kojav@kojav:~)$ gobuster -h
Usage.
gobuster [command]

Available Commands:
completion  Generate the autocompletion script for the specified shell
dir         Uses directory/file enumeration mode
dns         Uses DNS subdomain enumeration mode
fuzz        Uses fuzzing mode. Replaces the keyword FUZZ in the URL, Headers and
the request body
gcs         Uses gcs bucket enumeration mode
help        Help about any command
s3          Uses aws bucket enumeration mode
tftp        Uses TFTP enumeration mode
version     shows the current version
vhost       Uses VHOST enumeration mode (you most probably want to use the IP address as the URL parameter)

Flags:
--debug Task 2 Enable debug output
--delay duration Time each thread waits between requests (e.g. 1500ms)
-h, --help Task 3 help for gobuster
--no-color Task 4 Disable color output
--no-error Task 4 Don't display errors
-z, --no-progress Task 5 Don't display progress
-o, --output string Output file to write results to (defaults to stdout)
-p, --pattern string File containing replacement patterns
-q, --quiet Task 5 Don't print the banner and other noise
-t, --threads int Number of concurrent threads (default 10)
-v, --verbose Task 5 Verbose output (errors)
-w, --wordlist string Path to the wordlist. Set to - to use STDIN.
--wordlist-offset int Resume from a given position in the wordlist (default 0)

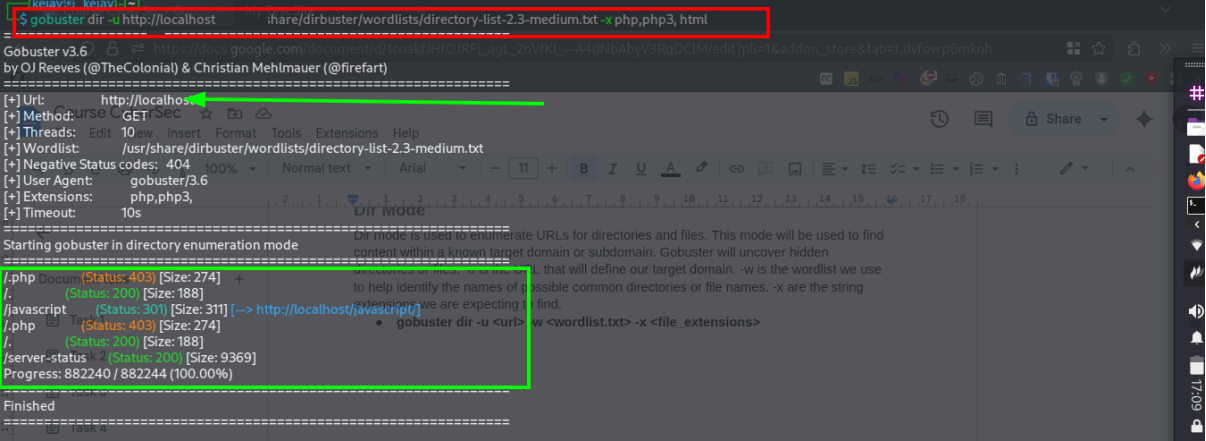
Use "gobuster [command] --help" for more information about a command.
```

As we can see in the output, Gobuster has **7 different brute force attack modes**, but we will only be using the 'dir' mode, which stands for directory brute forcing.

Dir Mode

Dir mode is used to enumerate URLs for directories and files. This mode will be used to find content within a known target domain or subdomain. Gobuster will uncover hidden directories or files. -u is the URL that will define our target domain. -w is the wordlist we use to help identify the names of possible common directories or file names. -x are the string extensions we are expecting to find.

- **gobuster dir -u <url> -w <wordlist.txt> -x <file_extensions>**



```
kelavie@kelavie: ~$ gobuster dir -u http://localhost -w /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt -x php,php3,html
Gobuster v3.6 by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://localhost
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt
[+] Negative status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php,php3,html
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/./php (Status: 403) [Size: 274]
/./ (Status: 200) [Size: 188]
/./javascript (Status: 301) [Size: 311] [-> http://localhost/javascript]
/./php (Status: 403) [Size: 274]
/./ (Status: 200) [Size: 188]
/./server-status (Status: 200) [Size: 9369]
Progress: 882240 / 882244 (100.00%)
Finished
=====
```

In the output, we can see that I first ran an Apache server on my laptop, and then executed Gobuster against my localhost. For this test, I used the medium wordlist from SecLists. I also specified in the command that Gobuster should only brute force files with extensions like **.php**, **.php3**, or **.html**. From the results, Gobuster discovered four files:

- **.php** files returned a **403 status code**, which means the files exist on the server but are restricted for normal or external users.
- **javascript** file returned a **301 status code**, indicating that the file is redirecting to another location.
- The **server-status** file returned a **200 status code**, meaning it is accessible and available to any user.

This demonstrates how directory brute forcing helps us expand the attack surface by discovering hidden directories and files.

Another important point is choosing the right wordlist. The wordlist should not be random; it should be selected carefully based on the target. For example, if the application is running on Ruby, we should use a Ruby-specific wordlist. If it is an educational site, we can use an education-related wordlist instead of a generic Ecommerce or Tech site wordlist.

For even better results, we can create our own custom wordlist. Since we already know what the website looks like and how it functions, we can gather possible directory names such as employee names, email IDs, or any other relevant words. We then save these words line by line in a **.txt** file and pass it to Gobuster using the **-w** flag. Gobuster will then use this custom wordlist to perform directory brute forcing against the target website.

In short, directory brute forcing is all about systematically guessing and discovering hidden files or directories to widen the attack surface.

Conclusion

Directory brute-forcing is an essential technique in penetration testing that helps discover hidden files and directories which may expose sensitive information. Tools like Gobuster and DIRB make this process efficient, and the right choice of wordlists significantly improves results. Overall, this task showed how directory enumeration widens the attack surface and highlights potential vulnerabilities to secure before attackers exploit them.